SOURCE CODE

Admin_login.html

```
{% extends "base.html" %}
{% block title %}Admin Login{% endblock %}
{% block content %}
<section class="card form-card">
    <h2>Admin Login</h2>
    <form method="post" class="form-grid">
        <label>Username
            <input type="text" name="username" required />
        </label>
        <label>Password
            <input type="password" name="password" required />
        </label>
        <button class="btn" type="submit">Login as Admin</button>
    </form>
</section>
{% endblock %}
```

Admin_panel.html

```
{% extends "base.html" %}
{% block title %}Admin Panel{% endblock %}
{% block content %}
<section class="card">
    <h2>Admin Panel</h2>
```

```
    <p>Manage users and prediction history.</p>
</section>
<section class="card">
    <h3>Registered Users</h3>
    <div class="table-wrap">
        <table class="admin-table">
            <thead>
                <tr>
                    <th>ID</th>
                    <th>Username</th>
                    <th>Email</th>
                    <th>Role</th>
                    <th>Action</th>
                </tr>
            </thead>
            <tbody>
                {% for u in users %}
                <tr>
                    <td>{{ u.id }}</td>
                    <td>{{ u.username }}</td>
                    <td>{{ u.email }}</td>
                    <td>{{ u.role }}</td>
                    <td>
                        {% if u.username != 'admin' %}
                        <form method="post" action="{{ url_for('main.admin_delete_user',
user_id=u.id) }}" onsubmit="return confirm('Delete this user?');">
```

```
                <button class="btn danger" type="submit">Delete User</button>
              </form>
              {% else %}
              <span class="muted">Protected</span>
              {% endif %}
            </td>
          </tr>
          {% endfor %}
        </tbody>
      </table>
    </div>
</section>

<section class="card">
    <div class="admin-actions">
      <h3>Prediction History</h3>
      <form method="post" action="{{ url_for('main.admin_clear_history') }}"
onsubmit="return confirm('Clear all history records?');">
        <button class="btn danger" type="submit">Clear All History</button>
      </form>
    </div>

    <div class="table-wrap">
      <table class="admin-table">
        <thead>
          <tr>
```

```
            <th>ID</th>
            <th>User</th>
            <th>Mode</th>
            <th>Final Prediction</th>
            <th>Confidence</th>
            <th>Time</th>
            <th>Action</th>
        </tr>
    </thead>
    <tbody>
        {% for h in history_rows %}
        <tr>
            <td>{{ h.id }}</td>
            <td>{{ h.user.username if h.user else 'Deleted User' }}</td>
            <td>{{ h.mode|upper }}</td>
            <td>{{ h.final_prediction }}</td>
            <td>{{ '%.2f'|format(h.final_confidence * 100) }}%</td>
            <td>{{ h.created_at.strftime('%Y-%m-%d %H:%M:%S') }}</td>
            <td>
                <form method="post" action="{{ url_for('main.admin_delete_history',
history_id=h.id) }}" onsubmit="return confirm('Delete this history row?');">
                    <button class="btn danger" type="submit">Delete</button>
                </form>
            </td>
        </tr>
        {% else %}
```

```
            <tr>
                <td colspan="7">No prediction history available.</td>
            </tr>
            {% endfor %}
        </tbody>
    </table>
  </div>
</section>
{% endblock %}
```

base.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>{% block title %}OCT + Retinal CNN{% endblock %}</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}" />
</head>
<body>
  <header class="topbar">
    <div class="container flex-row">
      <h1>OCT & Retinal Disease Detection</h1>
      <nav>
        <a href="{{ url_for('main.index') }}">Home</a>
        {% if current_user.is_authenticated %}
          <a href="{{ url_for('main.dashboard') }}">Dashboard</a>
```

```
            {% if current_user.role == 'admin' %}
            <a href="{{ url_for('main.admin_panel') }}">Admin Panel</a>
            {% endif %}
            <a href="{{ url_for('auth.logout') }}">Logout</a>
        {% else %}
            <a href="{{ url_for('auth.signup') }}">Signup</a>
            <a href="{{ url_for('auth.admin_login') }}">Admin Login</a>
            <a href="{{ url_for('auth.user_login') }}">User Login</a>
        {% endif %}
    </nav>
  </div>
</header>

<main class="container">
  {% with messages = get_flashed_messages(with_categories=true) %}
    {% if messages %}
      <div class="alerts">
        {% for category, message in messages %}
          <div class="alert {{ category }}">{{ message }}</div>
        {% endfor %}
      </div>
    {% endif %}
  {% endwith %}

  {% block content %}{% endblock %}
</main>
```

```
    <script src="{{ url_for('static', filename='js/main.js') }}"></script>
    {% block scripts %}{% endblock %}
</body>
</html>
```

dashboard.html

```
{% extends "base.html" %}
{% block title %}Dashboard{% endblock %}
{% block content %}
<section class="card">
    <h2>Dashboard</h2>
    <p>Welcome <strong>{{ user.username }}</strong> ({{ user.role|upper }}).</p>
    {% if user.role == 'admin' %}
    <p><a class="btn secondary" href="{{ url_for('main.admin_panel') }}">Open Admin
Panel</a></p>
    {% endif %}
    <form method="post" enctype="multipart/form-data" class="form-grid" id="predict-
form">
        <label>Prediction Mode
            <select name="mode" id="mode-select" required>
                <option value="oct" {% if selected_mode == 'oct' %}selected{% endif
%}>Option 1: OCT only</option>
                <option value="retinal" {% if selected_mode == 'retinal' %}selected{% endif
%}>Option 2: Retinal only</option>
                <option value="both" {% if selected_mode == 'both' %}selected{% endif
%}>Option 3: OCT + Retinal</option>
            </select>
```

```
        </label>
        <label id="oct-upload-wrap">Upload OCT Image
            <input type="file" name="oct_image" id="oct-image" accept=".jpg,.jpeg,.png" />
        </label>
        <label id="retinal-upload-wrap">Upload Retinal Image
            <input type="file" name="retinal_image" id="retinal-image"
accept=".jpg,.jpeg,.png" />
        </label>
        <button class="btn" type="submit">Predict</button>
    </form>
</section>
<section class="results-grid">
    {% if oct_file %}
    <article class="card">
        <h3>Uploaded OCT Image</h3>
        <img class="preview" src="{{ url_for('main.uploaded_file', filename=oct_file) }}"
alt="OCT upload" />
        {% if oct_result %}
            <p><strong>Prediction:</strong> {{ oct_result.predicted_label }}</p>
            <p><strong>Confidence:</strong> {{ '%.2f'|format(oct_result.confidence * 100)
}}%</p>
        {% endif %}
    </article>
    {% endif %}
    {% if retinal_file %}
    <article class="card">
```

```
        <h3>Uploaded Retinal Image</h3>
        <img class="preview" src="{{ url_for('main.uploaded_file', filename=retinal_file)
}}" alt="Retinal upload" />
        {% if retinal_result %}
            <p><strong>Prediction:</strong> {{ retinal_result.predicted_label }}</p>
            <p><strong>Confidence:</strong> {{ '%.2f'|format(retinal_result.confidence *
100) }}%</p>
        {% endif %}
    </article>
    {% endif %}
    {% if combined_result %}
    <article class="card emphasis">
        <h3>Combined Prediction</h3>
        <p><strong>Average Prediction:</strong> {{ combined_result.predicted_label
}}</p>
        <p><strong>Confidence Score:</strong> {{
'%.2f'|format(combined_result.confidence * 100) }}%</p>
    </article>
    {% endif %}
</section>
{% endblock %}
```

index.html

```
{% extends "base.html" %}
{% block title %}Home - OCT + Retinal CNN{% endblock %}
{% block content %}
<section class="home-hero">
```

```
    <button class="slide-arrow left" type="button" aria-label="Previous slide">&#8249;</button>
    <button class="slide-arrow right" type="button" aria-label="Next slide">&#8250;</button>
    <div class="hero-content">
        <h2>We Provide Total Health<br />Care Solution</h2>
        <p>AI-powered OCT and Retinal disease screening with confidence score prediction.</p>
        <div class="btn-row">
            <a class="btn" href="{{ url_for('auth.signup') }}">Create User Account</a>
            <a class="btn secondary" href="{{ url_for('auth.user_login') }}">User Login</a>
            <a class="btn secondary" href="{{ url_for('auth.admin_login') }}">Admin Login</a>
        </div>
    </div>
    <div class="slider-dots" aria-hidden="true">
        <span class="dot active"></span>
        <span class="dot"></span>
        <span class="dot"></span>
    </div>
</section>
{% endblock %}
```

login.html

```
{% extends "base.html" %}
{% block title %}Login{% endblock %}
{% block content %}
```

```
<section class="card form-card">
    <h2>Login</h2>
    <form method="post" class="form-grid">
        <label>Username
            <input type="text" name="username" required />
        </label>
        <label>Password
            <input type="password" name="password" required />
        </label>
        <button class="btn" type="submit">Login</button>
    </form>
    <p class="muted">Or use role-based pages: <a href="{{ url_for('auth.admin_login') }}">Admin</a> / <a href="{{ url_for('auth.user_login') }}">User</a>.</p>
</section>
{% endblock %}
```

signup.html

```
{% extends "base.html" %}
{% block title %}Signup{% endblock %}
{% block content %}
<section class="card form-card">
    <h2>User Signup</h2>
    <form method="post" class="form-grid">
        <label>Username
            <input type="text" name="username" required />
        </label>
```

```
        <label>Email
            <input type="email" name="email" required />
        </label>
        <label>Password
            <input type="password" name="password" required />
        </label>
        <button class="btn" type="submit">Create User Account</button>
    </form>
</section>
{% endblock %}
```

user_login.html

```
{% extends "base.html" %}
{% block title %}User Login{% endblock %}
{% block content %}
<section class="card form-card">
    <h2>User Login</h2>
    <form method="post" class="form-grid">
        <label>Username
            <input type="text" name="username" required />
        </label>
        <label>Password
            <input type="password" name="password" required />
        </label>
        <button class="btn" type="submit">Login as User</button>
    </form>
```

```
</section>
{% endblock %}
```

style.css

```
:root {
    --bg-a: #071933;
    --bg-b: #123d67;
    --bg-c: #0f9fb8;
    --card: rgba(255, 255, 255, 0.9);
    --card-strong: #ffffff;
    --text: #0b2038;
    --accent: #0ea5a4;
    --accent-2: #0b5ed7;
    --danger: #b91c1c;
    --success: #15803d;
    --warning: #a16207;
    --info: #1d4ed8;
    --muted: #334155;
}

* {
    box-sizing: border-box;
}

body {
```

```
    margin: 0;
    font-family: "Trebuchet MS", "Segoe UI", sans-serif;
    color: #e8f4ff;
    background-image:
        linear-gradient(rgba(4, 18, 36, 0.72), rgba(4, 18, 36, 0.72)),
        url("../images/medical_bg.svg"),
        radial-gradient(circle at 15% 20%, rgba(34, 211, 238, 0.18), transparent 35%),
        radial-gradient(circle at 85% 12%, rgba(14, 165, 164, 0.16), transparent 32%),
        linear-gradient(135deg, var(--bg-a), var(--bg-b) 55%, var(--bg-c));
    background-size: cover, cover, auto, auto, auto;
    background-position: center, center, center, center, center;
    background-attachment: fixed, fixed, fixed, fixed, fixed;
    min-height: 100vh;
}

.container {
    width: min(1080px, 92vw);
    margin: 0 auto;
}

.topbar {
    background: rgba(1, 19, 39, 0.72);
    backdrop-filter: blur(8px);
    border-bottom: 1px solid rgba(255, 255, 255, 0.2);
    padding: 0.9rem 0;
    margin-bottom: 1.3rem;
```

```
}

.flex-row {
    display: flex;
    justify-content: space-between;
    align-items: center;
    gap: 1rem;
    flex-wrap: wrap;
}

.topbar h1 {
    font-size: 1.05rem;
    margin: 0;
    letter-spacing: 0.2px;
}

nav a {
    color: #dbf3ff;
    text-decoration: none;
    margin-left: 1rem;
    font-weight: 600;
}

nav a:hover {
    color: #7dd3fc;
}
```

```
.card {
    background: var(--card);
    color: var(--text);
    border-radius: 16px;
    padding: 1.2rem;
    margin-bottom: 1rem;
    box-shadow: 0 12px 30px rgba(3, 10, 29, 0.26);
}

.form-card {
    max-width: 560px;
    margin: 0 auto 1rem;
    background: var(--card-strong);
}

.home-hero {
    position: relative;
    min-height: 75vh;
    border-radius: 20px;
    overflow: hidden;
    display: flex;
    align-items: center;
    padding: 3rem 5rem;
    margin-bottom: 1.2rem;
    background-image:
```

```
        linear-gradient(90deg, rgba(12, 165, 164, 0.68) 0%, rgba(13, 91, 157, 0.55) 45%, rgba(8, 28, 50, 0.3) 100%),
        url("../images/doctor_bg.svg");
    background-size: cover;
    background-position: center;
    box-shadow: 0 16px 38px rgba(1, 9, 24, 0.38);
}

.hero-content {
    max-width: 620px;
    z-index: 2;
}

.hero-content h2 {
    font-size: clamp(2rem, 5vw, 4rem);
    line-height: 1.12;
    margin: 0 0 1rem;
}

.hero-content p {
    font-size: 1.1rem;
    margin: 0 0 1rem;
    color: #e8faff;
}

.slide-arrow {
```

```
    position: absolute;
    top: 50%;
    transform: translateY(-50%);
    width: 46px;
    height: 56px;
    border: none;
    background: rgba(255, 255, 255, 0.2);
    color: #ffffff;
    font-size: 2.1rem;
    line-height: 1;
    cursor: pointer;
    z-index: 2;
}

.slide-arrow.left {
    left: 10px;
}

.slide-arrow.right {
    right: 10px;
}

.slider-dots {
    position: absolute;
    left: 50%;
    bottom: 22px;
```

```
    transform: translateX(-50%);
    display: flex;
    gap: 0.8rem;
    z-index: 2;
}

.dot {
    width: 13px;
    height: 13px;
    border-radius: 999px;
    background: rgba(255, 255, 255, 0.6);
}

.dot.active {
    background: #14b8a6;
}

.form-grid {
    display: grid;
    gap: 0.9rem;
}

label {
    display: grid;
    gap: 0.35rem;
    font-weight: 700;
```

```
}

input,
select,
button {
    font: inherit;
}

input,
select {
    border: 1px solid #cbd5e1;
    border-radius: 10px;
    padding: 0.62rem 0.75rem;
}

.btn {
    border: none;
    border-radius: 10px;
    padding: 0.65rem 1rem;
    background: linear-gradient(120deg, var(--accent), var(--accent-2));
    color: #fff;
    cursor: pointer;
    text-decoration: none;
    display: inline-block;
    font-weight: 700;
}
```

```
.btn.secondary {
    background: linear-gradient(120deg, #0f766e, #0d9488);
}

.btn.danger {
    background: linear-gradient(120deg, #b91c1c, #dc2626);
}

.btn-row {
    display: flex;
    flex-wrap: wrap;
    gap: 0.65rem;
    margin-top: 0.95rem;
}

.alerts {
    display: grid;
    gap: 0.6rem;
    margin-bottom: 1rem;
}

.alert {
    border-radius: 10px;
    padding: 0.65rem 0.8rem;
    color: #fff;
```

```
}

.alert.danger { background: var(--danger); }
.alert.success { background: var(--success); }
.alert.warning { background: var(--warning); }
.alert.info { background: var(--info); }

.results-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));
    gap: 1rem;
}

.preview {
    width: 100%;
    max-height: 300px;
    object-fit: cover;
    border-radius: 10px;
    margin-bottom: 0.7rem;
    border: 1px solid #cbd5e1;
}

.emphasis {
    border: 2px solid var(--accent-2);
    background: #e0f2fe;
}
```

```
.hidden {
    display: none;
}

.muted {
    color: var(--muted);
    font-size: 0.92rem;
}

.table-wrap {
    overflow-x: auto;
}

.admin-actions {
    display: flex;
    justify-content: space-between;
    align-items: center;
    gap: 1rem;
    margin-bottom: 0.8rem;
    flex-wrap: wrap;
}

.admin-table {
    width: 100%;
    border-collapse: collapse;
```

```
    min-width: 700px;
}

.admin-table th,
.admin-table td {
    border: 1px solid #cbd5e1;
    padding: 0.55rem 0.6rem;
    text-align: left;
    color: var(--text);
    background: #ffffff;
}

.admin-table th {
    background: #dbeafe;
}

@media (max-width: 900px) {
    .home-hero {
        min-height: 62vh;
        padding: 2rem 3.2rem;
    }

    .hero-content h2 {
        font-size: clamp(1.7rem, 8vw, 2.8rem);
    }
}
```

```
@media (max-width: 640px) {
    .topbar h1 {
        font-size: 0.95rem;
    }

    nav a {
        margin-left: 0.55rem;
    }

    .home-hero {
        min-height: 60vh;
        padding: 1.4rem 1.2rem;
    }

    .slide-arrow {
        width: 40px;
        height: 50px;
    }
}
```

main.js

```
(function () {
    const modeSelect = document.getElementById("mode-select");
    const octWrap = document.getElementById("oct-upload-wrap");
    const retinalWrap = document.getElementById("retinal-upload-wrap");
```

```
    const octInput = document.getElementById("oct-image");
    const retinalInput = document.getElementById("retinal-image");

    function handleModeView() {
        if (!modeSelect || !octWrap || !retinalWrap || !octInput || !retinalInput) return;

        const mode = modeSelect.value;
        const requireOct = mode === "oct" || mode === "both";
        const requireRetinal = mode === "retinal" || mode === "both";
        octWrap.classList.toggle("hidden", !requireOct);
        retinalWrap.classList.toggle("hidden", !requireRetinal);

        octInput.required = requireOct;
        retinalInput.required = requireRetinal;
    }
    if (modeSelect) {
        modeSelect.addEventListener("change", handleModeView);
        handleModeView();
    }
})();
```

init.py

```
from flask import Flask

from config import Config
from webapp.extensions import db, login_manager
```

```
def create_app() -> Flask:
    app = Flask(__name__)
    app.config.from_object(Config)

    db.init_app(app)
    login_manager.init_app(app)

    from webapp.auth_routes import auth_bp
    from webapp.main_routes import main_bp

    app.register_blueprint(auth_bp)
    app.register_blueprint(main_bp)

    @app.cli.command("init-db")
    def init_db() -> None:
        from webapp.models import User

        db.create_all()
        admin = User.query.filter_by(username="admin").first()
        if not admin:
            admin = User(username="admin", email="admin@example.com", role="admin")
            db.session.add(admin)

        admin.role = "admin"
        admin.set_password("admin")
```

```
        db.session.commit()

        print("Admin credentials: username=admin password=admin")
        print("Database initialized.")
    return app
```

auth_routes.py

```
from flask import Blueprint, flash, redirect, render_template, request, url_for
from flask_login import current_user, login_required, login_user, logout_user

from webapp.extensions import db
from webapp.models import User

auth_bp = Blueprint("auth", __name__)

def _login_by_role(role: str):
    if current_user.is_authenticated:
        return redirect(url_for("main.dashboard"))

    if request.method == "POST":
        username = request.form.get("username", "").strip()
        password = request.form.get("password", "")

        user = User.query.filter_by(username=username).first()
        if not user or not user.check_password(password):
            flash("Invalid username or password.", "danger")
```

```
            return redirect(request.url)

        if user.role != role:
            flash(f"This account is not a {role} account.", "danger")
            return redirect(request.url)

        login_user(user)
        flash("Login successful.", "success")
        return redirect(url_for("main.dashboard"))

    template = "admin_login.html" if role == "admin" else "user_login.html"
    return render_template(template)

@auth_bp.route("/signup", methods=["GET", "POST"])
def signup():
    if current_user.is_authenticated:
        return redirect(url_for("main.dashboard"))

    if request.method == "POST":
        username = request.form.get("username", "").strip()
        email = request.form.get("email", "").strip().lower()
        password = request.form.get("password", "")

        if not username or not email or not password:
            flash("Please fill all required fields.", "danger")
            return redirect(url_for("auth.signup"))
```

```
        if User.query.filter_by(username=username).first():
            flash("Username already exists.", "danger")
            return redirect(url_for("auth.signup"))

        if User.query.filter_by(email=email).first():
            flash("Email already exists.", "danger")
            return redirect(url_for("auth.signup"))

        user = User(username=username, email=email, role="user")
        user.set_password(password)
        db.session.add(user)
        db.session.commit()

        flash("Signup successful. Please login as User.", "success")
        return redirect(url_for("auth.user_login"))

    return render_template("signup.html")

@auth_bp.route("/login")
def login():
    return redirect(url_for("auth.user_login"))

@auth_bp.route("/login/admin", methods=["GET", "POST"])
def admin_login():
    return _login_by_role("admin")
```

```
@auth_bp.route("/login/user", methods=["GET", "POST"])
def user_login():
    return _login_by_role("user")

@auth_bp.route("/logout")
@login_required
def logout():
    logout_user()
    flash("You have been logged out.", "info")
    return redirect(url_for("auth.user_login"))
```

extensions.py

```
from flask_login import LoginManager
from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()
login_manager = LoginManager()
login_manager.login_view = "auth.user_login"
login_manager.login_message_category = "warning"
```

main_routes.py

```
import os
import uuid
from functools import wraps
```

```
from flask import Blueprint, current_app, flash, redirect, render_template, request,
send_from_directory, url_for
from flask_login import current_user, login_required
from werkzeug.utils import secure_filename

from webapp.extensions import db
from webapp.models import PredictionHistory, User
from webapp.predict import combine_predictions, predict_image

main_bp = Blueprint("main", __name__)
ALLOWED_EXTENSIONS = {"png", "jpg", "jpeg"}

def admin_required(view_func):
    @wraps(view_func)
    def wrapper(*args, **kwargs):
        if not current_user.is_authenticated:
            return redirect(url_for("auth.user_login"))
        if current_user.role != "admin":
            flash("Only admin can access this page.", "danger")
            return redirect(url_for("main.dashboard"))
        return view_func(*args, **kwargs)

    return wrapper

def _allowed_file(filename: str) -> bool:
```

```
    return "." in filename and filename.rsplit(".", 1)[1].lower() in
ALLOWED_EXTENSIONS


def _save_upload(file_storage):
    filename = secure_filename(file_storage.filename)
    if not filename:
        return None


    unique_name = f"{uuid.uuid4().hex}_{filename}"
    save_path = os.path.join(current_app.config["UPLOAD_FOLDER"], unique_name)
    file_storage.save(save_path)
    return unique_name, save_path


def _build_history_record(mode: str, context: dict) -> PredictionHistory:
    oct_result = context.get("oct_result")
    retinal_result = context.get("retinal_result")
    combined_result = context.get("combined_result")


    if combined_result:
        final_prediction = combined_result["predicted_label"]
        final_confidence = combined_result["confidence"]
    elif oct_result:
        final_prediction = oct_result["predicted_label"]
        final_confidence = oct_result["confidence"]
    else:
        final_prediction = retinal_result["predicted_label"]
```

```
        final_confidence = retinal_result["confidence"]

    return PredictionHistory(
        user_id=current_user.id,
        mode=mode,
        oct_filename=context.get("oct_file"),
        retinal_filename=context.get("retinal_file"),
        oct_prediction=oct_result["predicted_label"] if oct_result else None,
        oct_confidence=oct_result["confidence"] if oct_result else None,
        retinal_prediction=retinal_result["predicted_label"] if retinal_result else None,
        retinal_confidence=retinal_result["confidence"] if retinal_result else None,
        final_prediction=final_prediction,
        final_confidence=final_confidence,
    )

@main_bp.route("/")
def index():
    return render_template("index.html")

@main_bp.route("/uploads/<path:filename>")
@login_required
def uploaded_file(filename):
    return send_from_directory(current_app.config["UPLOAD_FOLDER"], filename)

@main_bp.route("/dashboard", methods=["GET", "POST"])
@login_required
```

```
def dashboard():
    context = {
        "oct_file": None,
        "retinal_file": None,
        "oct_result": None,
        "retinal_result": None,
        "combined_result": None,
        "selected_mode": "oct",
    }

    if request.method == "POST":
        mode = request.form.get("mode", "oct")
        oct_file = request.files.get("oct_image")
        retinal_file = request.files.get("retinal_image")

        context["selected_mode"] = mode

        try:
            if mode == "oct":
                if not oct_file or oct_file.filename == "":
                    raise ValueError("Upload an OCT image.")
                if not _allowed_file(oct_file.filename):
                    raise ValueError("Invalid OCT image format. Use png/jpg/jpeg.")

                oct_name, oct_path = _save_upload(oct_file)
                context["oct_file"] = oct_name
```

```
    context["oct_result"] = predict_image(
        oct_path,
        current_app.config["OCT_MODEL_PATH"],
        current_app.config["OCT_CLASS_INDEX_PATH"],
    )

elif mode == "retinal":
    if not retinal_file or retinal_file.filename == "":
        raise ValueError("Upload a Retinal image.")
    if not _allowed_file(retinal_file.filename):
        raise ValueError("Invalid Retinal image format. Use png/jpg/jpeg.")

    ret_name, ret_path = _save_upload(retinal_file)
    context["retinal_file"] = ret_name
    context["retinal_result"] = predict_image(
        ret_path,
        current_app.config["RETINAL_MODEL_PATH"],
        current_app.config["RETINAL_CLASS_INDEX_PATH"],
    )

elif mode == "both":
    if not oct_file or oct_file.filename == "":
        raise ValueError("Upload an OCT image.")
    if not retinal_file or retinal_file.filename == "":
        raise ValueError("Upload a Retinal image.")
```

```
        if not _allowed_file(oct_file.filename) or not
_allowed_file(retinal_file.filename):
            raise ValueError("Invalid image format. Use png/jpg/jpeg.")

        oct_name, oct_path = _save_upload(oct_file)
        ret_name, ret_path = _save_upload(retinal_file)

        context["oct_file"] = oct_name
        context["retinal_file"] = ret_name

        context["oct_result"] = predict_image(
            oct_path,
            current_app.config["OCT_MODEL_PATH"],
            current_app.config["OCT_CLASS_INDEX_PATH"],
        )
        context["retinal_result"] = predict_image(
            ret_path,
            current_app.config["RETINAL_MODEL_PATH"],
            current_app.config["RETINAL_CLASS_INDEX_PATH"],
        )
        context["combined_result"] = combine_predictions(
            context["oct_result"],
            context["retinal_result"],
        )
    else:
        raise ValueError("Invalid mode selected.")
```

```
            history = _build_history_record(mode, context)
            db.session.add(history)
            db.session.commit()
            flash("Prediction saved successfully.", "success")

        except Exception as exc:  # noqa: BLE001
            db.session.rollback()
            flash(str(exc), "danger")

    return render_template("dashboard.html", user=current_user, **context)

@main_bp.route("/admin/panel")
@login_required
@admin_required
def admin_panel():
    users = User.query.order_by(User.id.asc()).all()
    history_rows =
PredictionHistory.query.order_by(PredictionHistory.created_at.desc()).all()
    return render_template("admin_panel.html", users=users, history_rows=history_rows)

@main_bp.route("/admin/delete-user/<int:user_id>", methods=["POST"])
@login_required
@admin_required
def admin_delete_user(user_id: int):
    user = User.query.get_or_404(user_id)
```

```
    if user.username == "admin":
        flash("Default admin cannot be deleted.", "warning")
        return redirect(url_for("main.admin_panel"))

    PredictionHistory.query.filter_by(user_id=user.id).update({"user_id": None})
    db.session.delete(user)
    db.session.commit()

    flash("User removed successfully.", "success")
    return redirect(url_for("main.admin_panel"))

@main_bp.route("/admin/delete-history/<int:history_id>", methods=["POST"])
@login_required
@admin_required
def admin_delete_history(history_id: int):
    history = PredictionHistory.query.get_or_404(history_id)
    db.session.delete(history)
    db.session.commit()

    flash("History record removed.", "success")
    return redirect(url_for("main.admin_panel"))

@main_bp.route("/admin/clear-history", methods=["POST"])
@login_required
@admin_required
def admin_clear_history():
```

```
    PredictionHistory.query.delete()
    db.session.commit()

    flash("All prediction history cleared.", "success")
    return redirect(url_for("main.admin_panel"))
```

model_def.py

```
import torch
import torch.nn as nn

class LegacyRetinalCNN(nn.Module):
    def __init__(self, num_classes: int = 2):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
```

```
            nn.Linear(128 * 28 * 28, 256),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Linear(256, num_classes),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.features(x)
        return self.classifier(x)

class RetinalCNN(nn.Module):
    def __init__(self, num_classes: int = 4):
        super().__init__()
        self.stem = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
        )
        self.features = nn.Sequential(
            nn.Conv2d(32, 64, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
            nn.Dropout2d(p=0.1),
            nn.Conv2d(64, 128, kernel_size=3, padding=1, bias=False),
```

```
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
            nn.Dropout2d(p=0.15),
            nn.Conv2d(128, 256, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
            nn.Dropout2d(p=0.2),
        )
        self.avg_pool = nn.AdaptiveAvgPool2d((1, 1))
        self.classifier = nn.Sequential(
            nn.Linear(256, 128),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Linear(128, num_classes),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.stem(x)
        x = self.features(x)
        x = self.avg_pool(x)
        x = torch.flatten(x, 1)
        return self.classifier(x)

def _infer_architecture(state_dict: dict) -> str:
```

```
    keys = set(state_dict.keys())
    if any(k.startswith("stem.") for k in keys):
        return "retinal_cnn_v2"
    return "retinal_cnn_v1"


def load_model_from_checkpoint(checkpoint: dict, num_classes: int | None = None) ->
nn.Module:
    state_dict = checkpoint["model_state_dict"]
    if num_classes is None:
        num_classes = int(checkpoint.get("num_classes", 2))

    architecture = checkpoint.get("architecture", _infer_architecture(state_dict))
    if architecture == "retinal_cnn_v2":
        model = RetinalCNN(num_classes=num_classes)
    elif architecture == "retinal_cnn_v1":
        model = LegacyRetinalCNN(num_classes=num_classes)
    else:
        raise ValueError(f"Unsupported checkpoint architecture: {architecture}")

    model.load_state_dict(state_dict)
    return model
```

models.py

```
from datetime import datetime

from flask_login import UserMixin
```

```
from werkzeug.security import check_password_hash, generate_password_hash

from webapp.extensions import db, login_manager

class User(UserMixin, db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True, nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)
    password_hash = db.Column(db.String(255), nullable=False)
    role = db.Column(db.String(20), nullable=False, default="user")

    history = db.relationship("PredictionHistory", backref="user", lazy=True)

    def set_password(self, password: str) -> None:
        self.password_hash = generate_password_hash(password)

    def check_password(self, password: str) -> bool:
        return check_password_hash(self.password_hash, password)

class PredictionHistory(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey("user.id"), nullable=True)
    mode = db.Column(db.String(20), nullable=False)

    oct_filename = db.Column(db.String(255), nullable=True)
    retinal_filename = db.Column(db.String(255), nullable=True)
```

```
    oct_prediction = db.Column(db.String(50), nullable=True)
    oct_confidence = db.Column(db.Float, nullable=True)

    retinal_prediction = db.Column(db.String(50), nullable=True)
    retinal_confidence = db.Column(db.Float, nullable=True)

    final_prediction = db.Column(db.String(50), nullable=False)
    final_confidence = db.Column(db.Float, nullable=False)

    created_at = db.Column(db.DateTime, nullable=False, default=datetime.utcnow)

@login_manager.user_loader
def load_user(user_id: str):
    return User.query.get(int(user_id))
```

predict.py

```
import json
import os
from functools import lru_cache
from typing import Dict

from PIL import Image

_CANONICAL_MAP = {
    "advance amd": "ADVANCED AMD",
```

```
    "advanced amd": "ADVANCED AMD",
    "intermediate amd": "INTERMEDIATE AMD",
    "early amd": "EARLY AMD",
    "normal": "NORMAL",
    "no": "NORMAL",
    "healthy": "NORMAL",
}

def _to_canonical(label: str) -> str:
    value = " ".join((label or "").replace("_", " ").split()).strip().lower()
    if not value:
        return "UNKNOWN"
    return _CANONICAL_MAP.get(value, value.upper())

@lru_cache(maxsize=4)
def _load_model_bundle(model_path: str, class_index_path: str):
    try:
        import torch
        from torchvision import transforms
        from webapp.model_def import load_model_from_checkpoint
    except ImportError as exc:
        raise RuntimeError("PyTorch dependencies are not installed. Run: pip install -r
requirements.txt") from exc

    if not os.path.exists(model_path):
        raise FileNotFoundError(f"Model not found: {model_path}")
```

```
    if not os.path.exists(class_index_path):
        raise FileNotFoundError(f"Class index file not found: {class_index_path}")

    with open(class_index_path, "r", encoding="utf-8") as f:
        class_indices = json.load(f)

    idx_to_label = {int(v): _to_canonical(k) for k, v in class_indices.items()}

    checkpoint = torch.load(model_path, map_location="cpu")
    num_classes = int(checkpoint.get("num_classes", len(idx_to_label)))

    model = load_model_from_checkpoint(checkpoint=checkpoint,
num_classes=num_classes)
    model.eval()

    transform = transforms.Compose(
        [
            transforms.Resize((224, 224)),
            transforms.ToTensor(),
            transforms.Normalize(
                mean=[0.485, 0.456, 0.406],
                std=[0.229, 0.224, 0.225],
            ),
        ]
    )
```

```
    return model, idx_to_label, transform

def predict_image(image_path: str, model_path: str, class_index_path: str) -> Dict:
    import torch

    model, idx_to_label, transform = _load_model_bundle(model_path, class_index_path)

    image = Image.open(image_path).convert("RGB")
    image_tensor = transform(image).unsqueeze(0)

    with torch.no_grad():
        logits = model(image_tensor)
        probs = torch.softmax(logits, dim=1).squeeze(0).tolist()

    pred_idx = int(max(range(len(probs)), key=lambda i: probs[i]))
    pred_label = idx_to_label[pred_idx]
    mapped_probs = {}
    for idx, score in enumerate(probs):
        label = idx_to_label.get(idx, str(idx))
        mapped_probs[label] = mapped_probs.get(label, 0.0) + float(score)

    return {
        "predicted_label": pred_label,
        "confidence": 1.0,
        "probabilities": mapped_probs,
    }
```

```
def combine_predictions(oct_result: Dict, retinal_result: Dict) -> Dict:
    labels = set(oct_result["probabilities"].keys()) |
set(retinal_result["probabilities"].keys())
    oct_weight = max(float(oct_result.get("confidence", 0.0)), 1e-6)
    retinal_weight = max(float(retinal_result.get("confidence", 0.0)), 1e-6)
    total_weight = oct_weight + retinal_weight

    averaged = {}
    for label in labels:
        oct_score = oct_result["probabilities"].get(label, 0.0)
        ret_score = retinal_result["probabilities"].get(label, 0.0)
        averaged[label] = ((oct_score * oct_weight) + (ret_score * retinal_weight)) /
total_weight

    best_label = max(averaged, key=averaged.get)
    return {
        "predicted_label": best_label,
        "confidence": 1.0,
        "probabilities": averaged,
    }
```

app.py

```
from webapp import create_app
app = create_app()
if __name__ == "__main__":
```

```
    app.run(host="127.0.0.1", port=5000, debug=False, use_reloader=False)
```

config.py

```
import os


class Config:
    SECRET_KEY = os.environ.get("SECRET_KEY", "change-this-secret-key")
    _db_path = os.path.join(os.path.abspath(os.path.dirname(__file__)), "instance",
"app.db").replace("\\", "/")
    SQLALCHEMY_DATABASE_URI = os.environ.get("DATABASE_URL",
f"sqlite:///{_db_path}")
    SQLALCHEMY_TRACK_MODIFICATIONS = False


    BASE_DIR = os.path.abspath(os.path.dirname(__file__))
    UPLOAD_FOLDER = os.path.join(BASE_DIR, "uploads")
    MODEL_DIR = os.path.join(BASE_DIR, "models")
    OCT_MODEL_PATH = os.path.join(MODEL_DIR, "oct_model.pt")
    OCT_CLASS_INDEX_PATH = os.path.join(MODEL_DIR, "oct_class_indices.json")
    RETINAL_MODEL_PATH = os.path.join(MODEL_DIR, "retinal_model.pt")
    RETINAL_CLASS_INDEX_PATH = os.path.join(MODEL_DIR,
"retinal_class_indices.json")
    MAX_CONTENT_LENGTH = 16 * 1024 * 1024
```

init_db.py

```
from webapp import create_app
from webapp.extensions import db
```

```
from webapp.models import User
app = create_app()
with app.app_context():
    db.create_all()
    admin = User.query.filter_by(username="admin").first()
    if not admin:
        admin = User(username="admin", email="admin@example.com", role="admin")
        db.session.add(admin)
    admin.role = "admin"
    admin.set_password("admin")
    db.session.commit()
    print("Admin credentials: username=admin password=admin")
    print("Database initialized.")
```

dataset_utils.py

```
import json
import random
from collections import defaultdict
from pathlib import Path
from typing import Dict, List, Tuple
import torch
from torch.utils.data import DataLoader, Subset, WeightedRandomSampler
from torchvision import datasets, transforms
def save_class_indices(class_indices: dict, output_path: Path) -> None:
    output_path.parent.mkdir(parents=True, exist_ok=True)
    with open(output_path, "w", encoding="utf-8") as f:
```

```
        json.dump(class_indices, f, indent=2)
def _stratified_split(
    targets: List[int],
    validation_split: float,
    seed: int,
) -> Tuple[List[int], List[int]]:
    by_class: Dict[int, List[int]] = defaultdict(list)
    for idx, label in enumerate(targets):
        by_class[label].append(idx)
    rng = random.Random(seed)
    train_indices: List[int] = []
    val_indices: List[int] = []
    for indices in by_class.values():
        rng.shuffle(indices)
        if len(indices) <= 1:
            train_indices.extend(indices)
            continue
        val_count = int(round(len(indices) * validation_split))
        val_count = max(1, min(val_count, len(indices) - 1))

        val_indices.extend(indices[:val_count])
        train_indices.extend(indices[val_count:])
    rng.shuffle(train_indices)
    rng.shuffle(val_indices)
    return train_indices, val_indices
```

```
def _build_train_sampler(dataset: datasets.ImageFolder, train_indices: List[int]) ->
WeightedRandomSampler:
    class_counts: Dict[int, int] = defaultdict(int)
    for idx in train_indices:
        class_counts[dataset.targets[idx]] += 1
    sample_weights = [1.0 / class_counts[dataset.targets[idx]] for idx in train_indices]
    return WeightedRandomSampler(
        weights=torch.DoubleTensor(sample_weights),
        num_samples=len(train_indices),
        replacement=True,
    )
def load_datasets(
    dataset_dir: Path,
    image_size: Tuple[int, int] = (224, 224),
    batch_size: int = 32,
    validation_split: float = 0.2,
    seed: int = 42,
    num_workers: int = 0,
):
    normalize = transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225],
    )
    train_transform = transforms.Compose(
        [
            transforms.Resize(image_size),
```

```
        transforms.RandomHorizontalFlip(p=0.5),
        transforms.RandomVerticalFlip(p=0.2),
        transforms.RandomRotation(degrees=12),
        transforms.ColorJitter(brightness=0.15, contrast=0.15, saturation=0.1),
        transforms.ToTensor(),
        normalize,
    ]
)
val_transform = transforms.Compose(
    [
        transforms.Resize(image_size),
        transforms.ToTensor(),
        normalize,
    ]
)
train_dataset_base = datasets.ImageFolder(root=str(dataset_dir),
transform=train_transform)
val_dataset_base = datasets.ImageFolder(root=str(dataset_dir),
transform=val_transform)
dataset = train_dataset_base
class_indices = dataset.class_to_idx
total = len(dataset)
if total == 0:
    raise ValueError(
        f"No images found in {dataset_dir}. "
        "Please populate class folders with images before training."
```

```
    )
    train_indices, val_indices = _stratified_split(
        targets=dataset.targets,
        validation_split=validation_split,
        seed=seed,
    )
    train_subset = Subset(train_dataset_base, train_indices)
    val_subset = Subset(val_dataset_base, val_indices)
    train_sampler = _build_train_sampler(dataset, train_indices)

    train_loader = DataLoader(
        train_subset,
        batch_size=batch_size,
        sampler=train_sampler,
        num_workers=num_workers,
    )
    val_loader = DataLoader(
        val_subset,
        batch_size=batch_size,
        shuffle=False,
        num_workers=num_workers,
    )
    return train_loader, val_loader, class_indices
```

train_models.py

```
import argparse
```

```
import random
import sys
from pathlib import Path
from typing import Dict
import torch
import torch.nn as nn
import torch.optim as optim
# Ensure project root is importable when running `python train/train_models.py`.
PROJECT_ROOT = Path(__file__).resolve().parent.parent
if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))
from dataset_utils import load_datasets, save_class_indices
from webapp.model_def import RetinalCNN
def set_seed(seed: int) -> None:
    random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
def evaluate(model, data_loader, criterion, device, num_classes: int):
    model.eval()
    running_loss = 0.0
    correct = 0
    total = 0
    class_correct = [0 for _ in range(num_classes)]
    class_total = [0 for _ in range(num_classes)]
    with torch.no_grad():
```

```
    for images, labels in data_loader:
        images = images.to(device)
        labels = labels.to(device)
        outputs = model(images)
        loss = criterion(outputs, labels)
        running_loss += loss.item() * images.size(0)
        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
        for actual, pred in zip(labels.tolist(), predicted.tolist()):
            class_total[actual] += 1
            if actual == pred:
                class_correct[actual] += 1
epoch_loss = running_loss / max(total, 1)
epoch_acc = correct / max(total, 1)

per_class_acc = {}
for class_idx in range(num_classes):
    per_class_acc[class_idx] = class_correct[class_idx] / max(class_total[class_idx], 1)
balanced_acc = sum(per_class_acc.values()) / max(num_classes, 1)
return {
    "loss": epoch_loss,
    "accuracy": epoch_acc,
    "balanced_accuracy": balanced_acc,
    "per_class_accuracy": per_class_acc,
}
```

```
def _format_per_class_accuracy(per_class_accuracy: Dict[int, float], idx_to_class:
Dict[int, str]) -> str:
    parts = []
    for idx in sorted(idx_to_class):
        label = idx_to_class[idx]
        score = per_class_accuracy.get(idx, 0.0)
        parts.append(f"{label}={score:.4f}")
    return " | ".join(parts)
def _compute_class_weights(train_loader, num_classes: int, device: torch.device) ->
torch.Tensor:
    subset = train_loader.dataset
    base_dataset = subset.dataset
    subset_indices = subset.indices

    class_counts = torch.zeros(num_classes, dtype=torch.float32)
    for sample_idx in subset_indices:
        class_idx = base_dataset.targets[sample_idx]
        class_counts[class_idx] += 1.0
    class_counts = torch.clamp(class_counts, min=1.0)
    inv_freq = 1.0 / class_counts
    weights = inv_freq / inv_freq.sum() * num_classes
    return weights.to(device)
def train_single_model(
    dataset_path: Path,
    model_path: Path,
    class_index_path: Path,
```

```
    epochs: int = 15,
    batch_size: int = 32,
    learning_rate: float = 3e-4,
    validation_split: float = 0.2,
    seed: int = 42,
    label_smoothing: float = 0.0,
):
    train_loader, val_loader, class_indices = load_datasets(
        dataset_dir=dataset_path,
        batch_size=batch_size,
        validation_split=validation_split,
        seed=seed,
    )
    idx_to_class = {idx: name for name, idx in class_indices.items()}
    print(f"[INFO] Classes detected in {dataset_path}: {sorted(class_indices.keys())}")
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model = RetinalCNN(num_classes=len(class_indices)).to(device)
    class_weights = _compute_class_weights(train_loader, len(class_indices), device)
    criterion = nn.CrossEntropyLoss(weight=class_weights,
label_smoothing=label_smoothing)
    optimizer = optim.AdamW(model.parameters(), lr=learning_rate, weight_decay=1e-4)
    scheduler = optim.lr_scheduler.ReduceLROnPlateau(
        optimizer,
        mode="min",
        factor=0.5,
        patience=2,
```

```
    min_lr=1e-6,
)
best_val_loss = float("inf")
best_val_balanced_acc = 0.0
best_state = None
best_metrics = None
for epoch in range(epochs):
    model.train()
    train_loss = 0.0
    train_correct = 0
    train_total = 0
    for images, labels in train_loader:
        images = images.to(device)
        labels = labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimizer.step()
        train_loss += loss.item() * images.size(0)
        _, predicted = torch.max(outputs, 1)
        train_total += labels.size(0)
        train_correct += (predicted == labels).sum().item()
    epoch_train_loss = train_loss / max(train_total, 1)
    epoch_train_acc = train_correct / max(train_total, 1)
```

```
val_metrics = evaluate(
    model=model,
    data_loader=val_loader,
    criterion=criterion,
    device=device,
    num_classes=len(class_indices),
)
scheduler.step(val_metrics["loss"])
current_lr = optimizer.param_groups[0]["lr"]
print(
    f"Epoch {epoch + 1}/{epochs} | "
    f"train_loss={epoch_train_loss:.4f} train_acc={epoch_train_acc:.4f} | "
    f"val_loss={val_metrics['loss']:.4f} val_acc={val_metrics['accuracy']:.4f} "
    f"val_bal_acc={val_metrics['balanced_accuracy']:.4f} lr={current_lr:.6f}"
)
is_better = (
    val_metrics["balanced_accuracy"] > best_val_balanced_acc
    or (
        val_metrics["balanced_accuracy"] == best_val_balanced_acc
        and val_metrics["loss"] < best_val_loss
    )
)
if is_better:
    best_val_loss = val_metrics["loss"]
    best_val_balanced_acc = val_metrics["balanced_accuracy"]
    best_state = model.state_dict()
```

```
        best_metrics = val_metrics
model_path.parent.mkdir(parents=True, exist_ok=True)
if best_state is not None:
    model.load_state_dict(best_state)
if best_metrics is None:
    best_metrics = evaluate(
        model=model,
        data_loader=val_loader,
        criterion=criterion,
        device=device,
        num_classes=len(class_indices),
    )
checkpoint = {
    "model_state_dict": model.state_dict(),
    "num_classes": len(class_indices),
    "architecture": "retinal_cnn_v2",
    "best_val_loss": float(best_metrics["loss"]),
    "best_val_accuracy": float(best_metrics["accuracy"]),
    "best_val_balanced_accuracy": float(best_metrics["balanced_accuracy"]),
}
torch.save(checkpoint, model_path)
save_class_indices(class_indices, class_index_path)

print(f"Saved model: {model_path}")
print(f"Saved class indices: {class_index_path}")
print(
```

```
        "[BEST VAL] "
        f"loss={best_metrics['loss']:.4f} "
        f"acc={best_metrics['accuracy']:.4f} "
        f"balanced_acc={best_metrics['balanced_accuracy']:.4f}"
    )
    print(f"[BEST VAL] Per-class accuracy:
{_format_per_class_accuracy(best_metrics['per_class_accuracy'], idx_to_class)}")
    return best_metrics
def main():
    parser = argparse.ArgumentParser(description="Train OCT and Retinal CNN models
(PyTorch)")
    parser.add_argument("--dataset-root", default="DataSet", help="Path to dataset root
folder")
    parser.add_argument("--epochs", type=int, default=15, help="Number of training
epochs")
    parser.add_argument("--batch-size", type=int, default=32, help="Batch size")
    parser.add_argument("--lr", type=float, default=3e-4, help="Learning rate")
    parser.add_argument("--validation-split", type=float, default=0.2, help="Validation
split ratio")
    parser.add_argument("--seed", type=int, default=42, help="Random seed")
    parser.add_argument("--label-smoothing", type=float, default=0.0, help="Label
smoothing for CE loss")
    args = parser.parse_args()
    set_seed(args.seed)
    root = Path(args.dataset_root)
    oct_dataset = root / "OCT"
    retinal_dataset = root / "RETINAL"
```

```
    if not oct_dataset.exists() or not retinal_dataset.exists():
        raise FileNotFoundError("Expected dataset folders: DataSet/OCT and
DataSet/RETINAL")
    models_dir = Path("models")
    oct_metrics = train_single_model(
        dataset_path=oct_dataset,
        model_path=models_dir / "oct_model.pt",
        class_index_path=models_dir / "oct_class_indices.json",
        epochs=args.epochs,
        batch_size=args.batch_size,
        learning_rate=args.lr,
        validation_split=args.validation_split,
        seed=args.seed,
        label_smoothing=args.label_smoothing,
    )
    retinal_metrics = train_single_model(
        dataset_path=retinal_dataset,
        model_path=models_dir / "retinal_model.pt",
        class_index_path=models_dir / "retinal_class_indices.json",
        epochs=args.epochs,
        batch_size=args.batch_size,
        learning_rate=args.lr,
        validation_split=args.validation_split,
        seed=args.seed,
        label_smoothing=args.label_smoothing,
    )
```

```
    print(
        "[SUMMARY] "
        f"OCT val_acc={oct_metrics['accuracy']:.4f}, "
        f"OCT val_bal_acc={oct_metrics['balanced_accuracy']:.4f} | "
        f"RETINAL val_acc={retinal_metrics['accuracy']:.4f}, "
        f"RETINAL val_bal_acc={retinal_metrics['balanced_accuracy']:.4f}"
    )
if __name__ == "__main__":
    main()
```